Programmation orientée objet (UAA14)

Exercices : série 1

Objets d'apprentissage :

- ✔ Modéliser une logique de programmation orientée objet.
- ✔ Déclarer une classe.
- ✔ Instancier une classe.
- ✔ Utiliser les méthodes de l'objet instancié.
- ✔ Traduire un algorithme dans un langage de programmation.
- ✔ Commenter des lignes de code.
- ✔ Tester le programme conçu.
- ✔ Caractériser les attributs dans une classe (encapsulation).
- ✔ Caractériser les méthodes dans une classe (encapsulation).
- ✔ Décrire la création d'un constructeur.
- ✔ Extraire d'un cahier des charges les informations nécessaires à la programmation.
- ✔ Programmer en recourant aux instructions et types de données nécessaires au développement d'une application.
- ✔ Corriger un programme défaillant.
- ✔ Développer une classe sur la base d'un cahier des charges en respectant le paradigme de la programmation orientée objet.
- ✔ Programmer en recourant aux classes nécessaires au développement d'une application orientée objet.
- ✔ Améliorer un programme pour répondre à un besoin défini.

Exercice 1 : créer une classe

Etape 1

Crée une classe nommée `Individu`. Cette classe doit comporter les attributs suivants :

- un prénom ;
- un age ;
- un sexe (M ou F) ;
- un domicile (nom de la ville uniquement)

Etape 2

Crée un constructeur pour la classe `Individu`.

Etape 3

Teste ton code :

- déclare un objet de type `Individu` nommé `moi` ;
- initialise la variable `moi` en utilisant le constructeur avec les valeurs te décrivent ;
- affiche le prénom, le sexe et l'âge de `moi` ;
- modifie le sexe de `moi` en W et l'âge en -124 ;
- affiche de nouveau le prénom, le sexe et l'âge de `moi` ;

Exercice 2 : la sécurité avant tout !

Etape 1

En les laissant publics (visibilité par défaut), les attributs de la classe `Individu` ne sont pas protégés : le « monde extérieur » (d'autres scripts PHP) peut les modifier à sa guise, en y mettant parfois des valeurs non autorisées.

Pour résoudre ce problème, transforme les attributs de la classe `Individu` en attributs privés. Observe que le serveur lance une erreur…

Si rien ne s'affiche, c'est que PHP a désactivé l'affichage des erreurs à l'écran. Pour le réactiver, ajoute ces trois lignes au début de ton script :

```php
ini_set('display_errors', '1');
ini_set('display_startup_errors', '1');
error_reporting(E_ALL);
```



Une convention importante à la programmation O.O est que les variables d'instances d'une classe doivent toujours être privées.

Etape 2

Le fait de mettre un attribut privé empêche toute modification du monde extérieur. Ils sont maintenants bien protégés… mais un peu trop !

Pour permettre la modification des attributs privés d'une classe, il faut utiliser un *getter* et un *setter* : des méthodes qui servent de « gardiens » et qui se chargent de donner les accès en lecture / écriture aux attributs, tout en les protégeant.

Ensuite, définis les méthodes magiques `__get` et `__set`:

- si on tente de mettre un sexe autre que M ou F, ce setter doit ignorer la modification
- si on tente de mettre un âge négatif ou supérieur à 120, ce setter doit ignorer la modification

Enfin, corrige ton programme afin d'utiliser les getters / setters. Tente plusieurs modifications autorisées et non autorisées et observe le résultat.

Exercice 3 : des accesseurs plus courts

Reprends la classe `Individu` telle que tu l'as laissée à la fin de l'exercice précédent.

Etape 1

À l'exercice précédent, tu avais le choix entre des accesseurs classiques (`getPrénom`, `setAge`…) et les méthodes magiques `__get` / `__set`. Si tu as choisi les accesseurs classiques, écris-les d'abord sous forme de `__get` et `__set` : la suite de l'exercice en a besoin.

Cela fait, compte le nombre de lignes qu'occupent ces deux méthodes pour quatre variables d'instance. Combien en faudrait-il si la classe `Individu` en comptait douze ? Et si on ajoutait une v.i six mois plus tard, combien d'endroits faudrait-il modifier ?

Etape 2

En PHP, le **nom** d'une variable d'instance peut lui-même être rangé dans une variable. Ajoute ces deux lignes à ton programme et exécute :

```php
$propriete = "prenom";
echo $moi->$propriete;
```

Qu'est-ce qui s'est affiché ? Remplace ensuite `"prenom"` par `"domicile"` et vérifie de nouveau.

`$moi->$propriete` va donc chercher la variable d'instance **dont le nom est contenu** dans `$propriete`.

Etape 3

Remplace tes deux méthodes par la version suivante, qui fonctionne pour n'importe quelle variable d'instance :

```php
function __get($propriete) {
    return $this->$propriete;
}

function __set($propriete, $valeur) {
    $this->$propriete = $valeur;
}
```

Vérifie que tous les affichages et toutes les modifications de l'exercice 2 fonctionnent encore.

Etape 4

Cette version est nettement plus courte… mais elle a perdu quelque chose en route. Exécute :

```php
$moi->couleurDesYeux = "bleu";
echo $moi->couleurDesYeux;

$moi->age = 250;
echo $moi->age;
```

La classe `Individu` ne possède pourtant aucune variable d'instance `couleurDesYeux`, et l'âge 250 aurait dû être refusé. Explique par écrit ce qui s'est passé dans chacun des deux cas.

Etape 5

Rends à la classe le contrôle qu'elle a perdu, **sans** revenir à la cascade de `if`. Ajoute-lui une variable d'instance :

```php
private $proprietesAutorisees = ["prenom", "age", "sexe", "domicile"];
```

puis modifie `__get` et `__set` pour qu'ils ignorent purement et simplement toute propriété absente de cette liste. La fonction `in_array` te sera utile.

Etape 6

Enfin, remets dans `__set` les deux vérifications de l'exercice 2 :

- un âge doit rester compris entre 0 et 120 ;
- un sexe ne peut valoir que M ou F.

Tu auras besoin de deux `if`, mais uniquement pour ces vérifications : l'affectation, elle, reste générique. Les autres propriétés restent modifiables sans condition. Teste une modification autorisée, une modification interdite et une propriété qui n'existe pas.

À retenir : cette écriture générique est partout dans les applications web. Elle permet d'écrire une classe par table de la base de données sans réécrire quarante accesseurs à la main.

Exercice 4 : méthode simple et efficace

Etape 1

Ajoute à la classe `Individu` une méthode nommée `sePrésenter` qui affiche le texte suivant :

« Je m'appelle `<prénom>` et je suis âgé de `<âge>` an(s). Je réside à `<domicile>`. »

Pour être méticuleux, fais en sorte que le « s » de « an(s) » ne s'affiche que si l'âge est supérieur à 1. Fais en sorte également d'afficher « âgée » ou « âgé » selon les cas.

Teste ensuite cette méthode en l'appelant !

Etape 2

Commente la méthode `sePrésenter` et écris une nouvelle version de cette méthode : cette dernière doit accepter un entier comme argument : 1 pour l'affichage en français, 2 pour l'affichage en anglais et 3 pour l'affichage en néerlandais (pas d'affichage dans les autres cas).

Teste cette nouvelle version avec les entrées 0, 1, 2 et 3.

Etape 3

Dé-commente la première version de `sePrésenter`. Il y a donc maintenant deux méthodes avec le même nom, mais pas avec la même signature. Le code est-il accepté ?

Regarde attentivement les deux versions : une grosse partie de leur code est identique. Comment pourrais-tu faire pour n'en garder qu'une tout en permettant de ne pas renseigner d'argument à la méthode (français par défaut).

> ⓘ Le principe du « point de modification unique » est un principe de programmation qui vise à réutiliser, le plus possible, du code déjà présent dans un programme dans le but de n'avoir qu'un seul « endroit dans le code » à modifier lorsque c'est nécessaire.

Exercice 5 : la carte de fidélité

Règle du jeu

À partir d'ici, on ne te donne plus d'étapes. Ni la liste des variables d'instance, ni celle des méthodes : c'est à toi de les décider. Tu ne reçois que la situation à modéliser, quelques exigences, et le résultat attendu à l'écran.

La situation

Un magasin veut gérer les cartes de fidélité de ses clients. Une carte est au nom d'un client et porte un numéro. Elle cumule des points : chaque euro dépensé rapporte un point.

Le client peut échanger 100 points contre une récompense, ce qui les lui retire. Une carte neuve démarre à zéro point, mais le magasin doit également pouvoir créer une carte avec un solde de départ, lors de la reprise d'une ancienne carte.

Les exigences

- le nombre de points d'une carte ne peut jamais devenir négatif ;
- personne, en dehors de la classe, ne peut fixer directement le nombre de points ;
- ton programme de test doit démontrer ces deux points.

Le résultat attendu

Écris la classe, puis un programme de test qui produit exactement ceci :

```
Carte 1042 - Zoé Martin : 0 point(s)
Carte 1042 - Zoé Martin : 85 point(s)
Échange impossible : il manque 15 point(s).
Carte 1042 - Zoé Martin : 130 point(s)
Récompense échangée !
Carte 1042 - Zoé Martin : 30 point(s)
Carte 2077 - Malik Ben Ali : 60 point(s)
```

Pour vérifier

Une fois ton programme au point, relis-le et réponds :

- combien de tes variables d'instance sont privées ? pourquoi celles-là ?
- à quel endroit as-tu placé le contrôle « jamais négatif » ? pourquoi là plutôt qu'ailleurs ?
- le numéro d'une carte peut-il être modifié après sa création ? est-ce souhaitable ?
- quel argument de ton constructeur est facultatif, et pourquoi doit-il être le dernier ?

```php
<?php
class CarteFidelite {

    private $numero;
    private $client;
    private $points;

    public function __construct($numero, $client, $points = 0) {
        $this->numero = $numero;
        $this->client = $client;

        if ($points < 0) {
            $points = 0;
        }
        $this->points = $points;
    }

    public function __get($propriete) {
        return $this->$propriete;
    }

    public function depenser($montant) {
        if($montant > 0){
            $this->points = $this->points + $montant;
            echo "Récompense échangée !\n";
        }
    }

    public function echangerRecompense() {
        if ($this->points >= 100) {
            $this->points = $this->points - 100;
        }else{
            $diff = ($this->points - 100) * -1;
            echo "Echange impossible : il manque $diff points\n";
        }
    }

    public function decrire() {
        echo "Carte " . $this->numero . " - " . $this->client
             . " : " . $this->points . " point(s)\n";
    }
}

$carte = new CarteFidelite("1042", "Zoé Martin");
$carte2 = new CarteFidelite("2077", "Malik Ben Ali", 60);
$carte->decrire();
$carte->depenser(85);
$carte->decrire();
$carte->echangerRecompense();
$carte->depenser(45);
$carte->decrire();
$carte->echangerRecompense();
$carte->decrire();
$carte2->decrire();
```

Références

Les présents exercices ont été élaborés à l'aide des ressources suivantes :

- https://www.pierre-giraud.com/php-mysql-apprendre-coder-cours/introduction-programmation-orientee-objet/
- Cours de « Développement d'applications WEB », Hénallux (2019)